Transaction Management

EMS Project — Study Guide: Why, How, and Practical Design

Purpose: A practical reference for understanding and implementing transaction management in the Spring Boot EMS application.

1. What Is a Database Transaction?

A transaction is a logical unit of database work that should be treated as one operation. It can contain one database operation or several related operations. The goal is to prevent a business operation from leaving the database in an invalid intermediate state.

> **Core idea:** the required work succeeds and is committed, or the transaction is rolled back when it fails.

2. Why Do We Need Transactions?

An EMS employee operation can involve several related actions: checking an email, finding a department, creating/updating an employee, and saving the result. As business operations become more complex, a later failure must not leave earlier database changes in an incorrect state.

> Transactions provide a controlled boundary around related database work and support atomicity.

3. ACID Properties

Atomicity: the transaction is treated as one unit. **Consistency:** a successful transaction leaves the database satisfying its rules. **Isolation:** concurrent transactions are controlled according to the database isolation behavior. **Durability:** committed changes are intended to persist.

4. Spring's @Transactional

In Spring Boot, **@Transactional** is commonly used at the service layer to define the transaction boundary of a business operation. Spring manages the transaction around the method invocation.

```
@Transactional
public EmployeeDTO createEmployee(EmployeeDTO employeeDTO) { ... }
```

For the EMS, the service layer is a natural place for transaction boundaries because it contains business operations rather than merely individual repository calls.

5. Commit and Rollback

Commit: the transaction completed successfully and its database changes are committed. **Rollback:** the transaction fails according to its rollback rules and its database changes are rolled back.

```
BEGIN TRANSACTION → database work → success → COMMIT
BEGIN TRANSACTION → database work → failure → ROLLBACK
```

6. Which EMS Methods Need Transactions?

Method	Recommendation	Reason
createEmployee()	@Transactional	Multiple related operations form one business operation.
updateEmployee()	@Transactional	Employee update and department resolution belong to one write operation.

Method	Recommendation	Reason
deleteEmployeeById()	@Transactional	A write operation with a clear transaction boundary.
getEmployeeById()	Usually no explicit transaction	Simple read; use one when the business operation needs a transactional pe
getAllEmployees()	Usually no explicit transaction	Simple read; use one when multiple related reads must form one consistent

7. Persistence Context and Consistent Reads

When a service method performs multiple related reads, one transaction can give the operation a defined transactional/persistence context. This is useful when those reads represent one consistent business operation. The visibility of concurrent changes is also affected by database isolation settings.

8. Concurrent Transactions

Transactions do not automatically mean that nobody else can access the same data. Concurrent transactions interact according to the database's isolation behavior. Isolation determines what one transaction can observe while another transaction is working.

> **Practical takeaway:** define meaningful transaction boundaries and introduce explicit concurrency-control mechanisms only when requirements justify them.

9. Propagation

The **propagation** attribute controls what a transactional method does when another transaction already exists. The default is **Propagation.REQUIRED**: join an existing transaction if one exists; otherwise create a new one.

```
@Transactional(propagation = Propagation.REQUIRED) is usually unnecessary in normal EMS code because REQUIRED is already the default.
```

10. REQUIRES_NEW

Propagation.REQUIRES_NEW suspends the current transaction and starts an independent transaction for the called method. It is appropriate only for specific use cases where independent transaction boundaries are genuinely required.

> Do not add REQUIRES_NEW simply to demonstrate propagation.

11. Rollback Rules

Spring's default rollback behavior generally rolls back for unchecked exceptions (RuntimeException and subclasses). Checked exceptions have different default behavior and can be configured explicitly.

> Example: @Transactional(rollbackFor = SomeCheckedException.class)

12. readOnly = true

A read-only transaction communicates that an operation is intended for reading and can allow suitable persistence/database optimizations. It is not a requirement for every GET method.

> Example: @Transactional(readOnly = true)

13. Self-Invocation

Spring commonly applies @Transactional through a proxy. An external call can pass through that proxy, while a direct method call from another method in the same class does not normally go through the proxy in the same way. This can make internal transactional calls surprising.

> Keep transaction boundaries around clear service operations and be aware of proxy-based behavior.

14. Transactions and Caffeine Cache

Your EMS now uses both database transactions and Caffeine caching. Therefore database state and cache state should remain logically consistent. Employee create/update/delete operations use cache eviction/update annotations alongside their database writes.

> When designing cache annotations, always consider what should happen if the database operation fails or rolls back. Caching is part of data-consistency design, not merely a performance feature.

15. Current EMS Transaction Design

The current design places transactions around employee write operations:

```
@Transactional
createEmployee()

@Transactional
updateEmployee()

@Transactional
deleteEmployeeById()
```

The read methods are not made transactional mechanically. The decision depends on whether their business semantics require a transaction or transactional persistence context.

16. What We Deliberately Did Not Add

Optimistic locking: useful when concurrent updates to the same entity are a real requirement, but not mandatory for the current EMS. We deliberately chose not to add it merely for feature count.

Pessimistic locking: should similarly be introduced only when a real business requirement needs database-level locking behavior.

17. Important Q&A;

Q: Why use @Transactional at the service layer?

A: The service layer represents business operations and can define one transaction boundary around multiple repository operations.

Q: What is commit?

A: Successful completion of a transaction, making its database changes committed.

Q: What is rollback?

A: Reversal of database changes made within the transaction according to its rollback rules.

Q: What is propagation?

A: The rule that determines how a method behaves when a transaction already exists.

Q: What is REQUIRED?

A: Join the existing transaction or create a new one if none exists. It is the default.

Q: What is REQUIRES_NEW?

A: Suspend the current transaction and execute the method in a new independent transaction.

Q: Should every GET have @Transactional?

A: No. Add it when the operation's business semantics require a transactional context or when an explicit read-only transaction is appropriate.

Q: What is readOnly=true?

A: A declaration that the transaction is intended for reads and may enable suitable optimizations.

Q: Why can self-invocation be a problem?

A: Because normal proxy-based Spring transaction interception is not applied to a direct internal call in the same way as an external call through the proxy.

Q: Why does caching matter to transactions?

A: Because database changes and cached values must remain logically consistent when operations succeed or fail.

18. Practical EMS Checklist

• Keep transaction boundaries at meaningful service-layer business operations.

• Use @Transactional for create/update/delete operations that form a business unit.

• Do not add @Transactional everywhere mechanically.

• Use readOnly transactions when they provide a clear semantic or performance benefit.

• Understand rollback behavior before designing exception handling.

• Remember that REQUIRED is the default propagation.

• Use REQUIRES_NEW only when an independent transaction is genuinely required.

• Be aware of Spring proxy/self-invocation behavior.

• Consider cache consistency whenever a transactional database write changes cached data.

• Test both successful commits and failure/rollback scenarios.

19. Final Mental Model

Think of a transaction as the boundary around a business operation:

```
HTTP Request → Controller → Service Transaction → Repository/JPA → Database
                                        ↓
                                  COMMIT or ROLLBACK
```

For the EMS, the goal is not to maximize the number of @Transactional annotations. The goal is to place transaction boundaries where they correctly represent business operations and data-consistency requirements.